MiniVerse

Full-Stack Working Roadmap

Frontend -> Game engine -> Database -> Local multiplayer -> Cloud servers
Deep analysis of the MiniVerse Build Explainer, gap fixes, end-to-end data flows, server API and schema, security, testing and a week-by-week build plan.

Android | Kotlin | Compose | Room | Wi-Fi Direct | Bluetooth | Ktor | PostgreSQL | Redis

One App. Endless Mini Adventures. | Powered by Devanshu | November 2026

Contents

1. Executive summary 3

2. Deep analysis of your guide 4

3. Full system architecture 6

4. Frontend - screens, state and rendering 8

5. Application layer - use cases and coordinators 10

6. The game engine in depth 12

7. Database and local storage 14

8. Local phone-to-phone connectivity 16

9. Server backend - connectivity to the cloud 19

10. Server data, sync and security 22

11. End-to-end flows - the whole app working 25

12. Project structure (client + server) 27

13. Performance, testing and release quality 28

14. Step-by-step roadmap 29

15. Decisions to confirm before coding 31

Appendix - Error codes and glossary 32

1. Executive summary

This document is the full working roadmap for **MiniVerse**: how the app works from the first pixel the player taps, through the ViewModel, the game engine, the local database, phone-to-phone connections, and finally the optional cloud servers. It is built on your *MiniVerse Complete Build Explainer* (32 pages), after a line-by-line analysis of it. Nothing in your guide is thrown away. This roadmap does three new things:

- **Fixes the gaps.** 14 issues found in the guide (contradictions, security holes in party games, a database rule that would block valid rewards, unclear multi-player wallets) - each with a concrete fix (section 2).
- **Adds the server layer.** Your guide is 100% offline. You asked for connectivity to servers, so this roadmap adds an **optional cloud backend** (accounts, cloud save, leaderboards, online play, remote config) that never breaks offline play (sections 9-10).
- **Connects every layer end to end.** Each data flow - first launch, a tap, a reward, a two-phone match, an online match, a cloud sync - is traced step by step across UI, engine, database and network (section 11).

The architecture in one sentence

A pure-Kotlin **game engine** decides everything; **Compose screens** only draw state and send taps; **Room** stores results and the coin ledger; a single **MultiplayerTransport** interface carries the same messages over pass-and-play, Wi-Fi Direct, Bluetooth or the internet; and an optional **Ktor server** runs *the very same engine module* to referee online matches and sync data across devices.

Key decisions this roadmap recommends

Area	Recommendation	Why
Core gameplay	Stays fully offline, as in your guide	Fast, private, no server bills to start
Backend language	Kotlin + Ktor server	Reuse core:gameengine on the server - one set of rules, zero drift
Server database	PostgreSQL (truth) + Redis (queues, live leaderboards, presence)	Relational data fits ledger/matches; Redis is ideal for real-time sets
Online transport	WebSocket over TLS, same message envelope as local	Engine and games need no changes for online play
Accounts	Anonymous device account first, optional Google sign-in to link	No login wall; players opt in to cloud save
Sync model	Outbox table + WorkManager, idempotent claim ids	Works after days offline; never double-pays coins
Release plan	v1.0 offline (8 games) in about 20 weeks, cloud v1.5 at about 30 weeks	Ship value early, add servers once the core is stable

> **How to use this roadmap**
> Read sections 1-3 once. Then build in the order of section 14, using sections 4-12 as the reference for each layer. Section 15 is the checklist of decisions to lock before writing code.

2. Deep analysis of your guide

2.1 What is already strong

- **Host-authoritative engine with a deterministic reduce()** - the single best decision in the guide. It makes testing, save/restore, local multiplayer and (now) online play all possible with one codebase.
- **Save first, animate second** for coins, inside one Room transaction - prevents the most common reward bug.
- **One transport interface** - lets us plug in an internet transport later without touching games.
- **Three sync families** (turn-based, real-time physics, score challenge) - correct and pragmatic.
- **Vertical-slice rule** (Tic Tac Toe end to end first) - the right way to de-risk the platform.

2.2 Issues found and the fix for each

#	Issue in the guide	Risk	Fix in this roadmap
1	Unique index on (matchId, playerId) for coins, but single-player gives several rewards per run (level + medal + personal best).	High - valid rewards silently rejected	Unique index on claimId = matchId:playerId:reason. See 7.3.
2	reduce() must be deterministic, yet dice, shuffles and spawns are random.	High - two phones disagree	Randomness enters only as data: host draws from a seeded RNG and puts the value inside the event (RollResult(4)). See 6.4.
3	Rock Paper Scissors: 'host reveals both choices' - but the host is also a player and sees the guest's choice first.	Medium - host can cheat	Commit-reveal: both send hash(choice + salt), then both reveal. See 8.6.
4	Guess the Word / Bomb Squad: host holds the secret, but the host may be a guesser.	Medium	Secret owner is the setter's device; UI never shows hidden data; accept casual trust offline, enforce on server online.
5	Same-device play with 2-4 people but only one profile and one wallet created at onboarding.	High - who gets the coins?	Decision D5: local guests are 'guest seats' (no wallet) or quick extra profiles. See 15.
6	Reaction Battle on two phones compares tap times across different clocks.	High - unfair results	Clock-offset handshake (NTP-style, 5 pings, keep lowest RTT). See 8.5.
7	Snakes and Ladders is 'Phase 2' in the catalogue but in the first-release 8 and built in weeks 5-8.	Low - confusion	Unified phase table in section 14.
8	Number Merge and Whack-a-Mole are Phase 5/6 in the catalogue but built in step 3.	Low	Unified phase table.
9	Build-order table says weeks 'with 3' for wallet (garbled).	Low	Wallet moves into the vertical slice (weeks 5-6).
10	Mini Ludo is Phase 1 but not in the first 8 games.	Low	Ludo moved to v1.1.
11	Guest explicitly quits a 2-phone match: rules only cover host loss and abandon.	Medium - rage-quit exploit	Forfeit rule: explicit quit after 30% of match = win for others; earlier = abandoned. Decision D6.
12	Real-time physics games cannot really resume from a checkpoint.	Low	Physics games: pause in RAM only; if the process dies, match = abandoned (no coins).

#	Issue in the guide	Risk	Fix in this roadmap
13	Permissions mention 'location' generally.	Medium - Play review	Per-version table: Android 12+ BLUETOOTH_SCAN/CONNECT/ADVERTISE, Android 13+ NEARBY_WIFI_DEVICES. See 8.8.
14	No server, no backup: uninstall or new phone = all coins and progress lost.	High for retention	Optional cloud save + account linking (section 9).

3. Full system architecture

MiniVerse has three zones. **Zone A** is everything inside one phone and works with no network at all. **Zone B** is the local link between nearby phones. **Zone C** is the optional cloud. The engine sits in the middle and is shared by Zone A and Zone C.

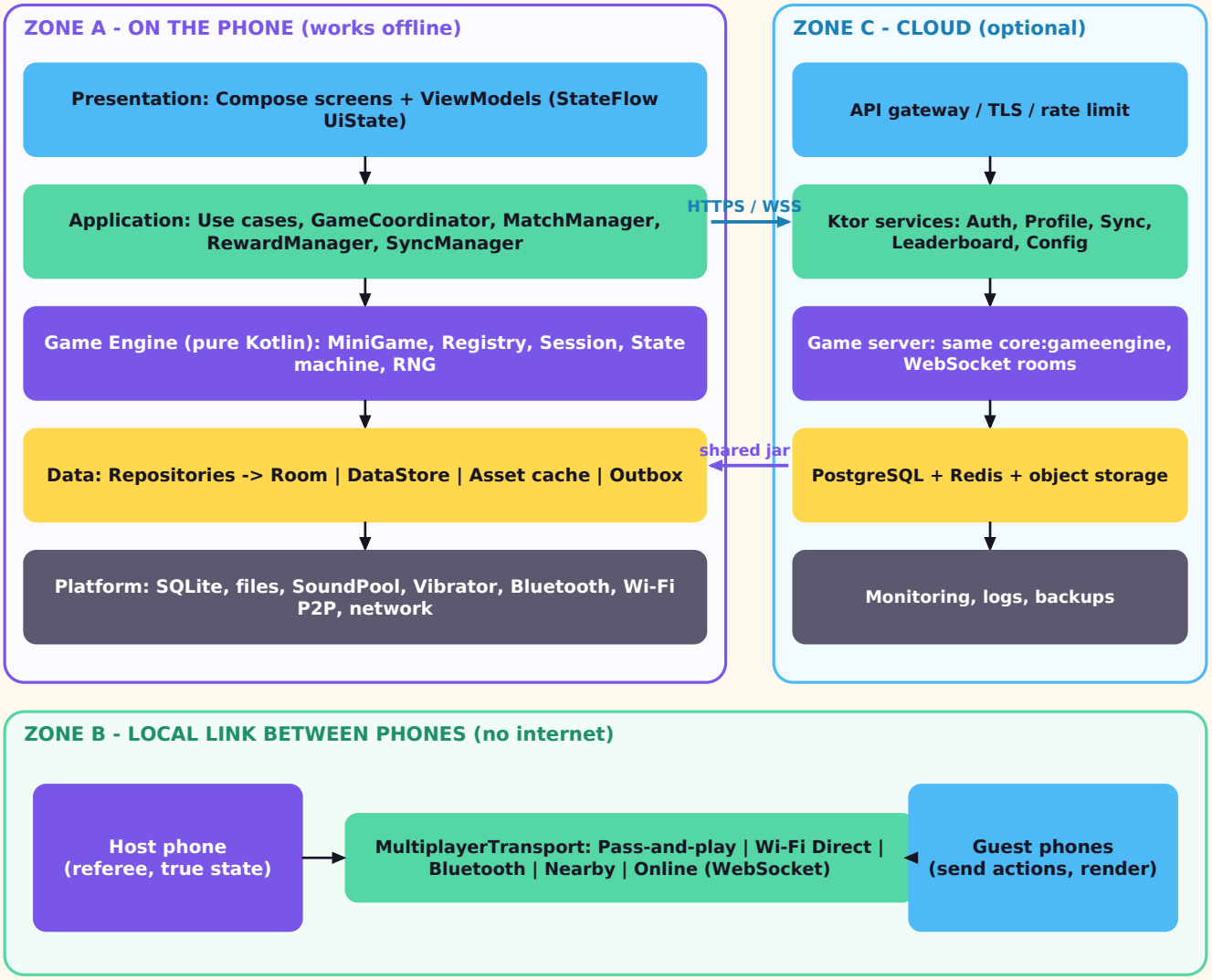


Figure 3.1 - The three zones. Zone C is optional: if it is down, Zones A and B keep working.

3.1 Responsibilities and dependency rule

Layer	Owns	Must never
Presentation (Compose)	Drawing state, capturing touches, animations, navigation calls	Decide a winner, compute coins, touch Room directly
ViewModel	UiState as StateFlow, one-shot Effects (navigate, snackbar), calling use cases	Hold Context or Views; run game loops
Application (use cases)	Orchestration: start match, apply action, finish match, award, sync	Contain game rules
Game engine	Rules, validation, reduce(), outcome, RNG from seed	Import Android, Room, network or wallet
Data / repositories	Room, DataStore, cache, outbox, network API clients	Contain game rules or UI text
Transport	Moving GameMessage envelopes between devices/server	Interpret game rules

Layer	Owns	Must never
Server	Accounts, sync, leaderboards, online refereeing	Be required for offline features

Dependency rule

Arrows only point downward: UI -> ViewModel -> use case -> engine/repository. The engine depends on nothing but core:model. Because of this, the server can import core:gameengine as a plain JVM library.

4. Frontend - screens, state and rendering

4.1 Screen map and navigation routes



Figure 4.1 - Main path. From Home: Wallet, Shop, Achievements, Profile, History, Settings, and (new) Leaderboards and Account.

Route	Screen	Arguments	ViewModel loads
splash	Splash	-	Theme, DB open, registry, profile, remote config cache
onboarding	Nickname + avatar	-	CreatePlayerUseCase (wallet + 100 MC claim 'starter')
home	Dashboard	-	Wallet flow, recents, featured, daily challenge
library?category=	Game grid	category, query	Registry list filtered, favourites flow
game/{gameId}	Details	gameId	GameDefinition, personal best, rules text
setup/{gameId}?mode=	Match setup	gameId, mode=LOCAL|HOST|JOIN|ONLINE	Seats, rounds, difficulty, permissions state
lobby/{sessionId}	Room lobby	sessionId	Transport state, peers, ready flags
play/{matchId}	Gameplay	matchId	Engine state flow, input lock, timers
result/{matchId}	Result	matchId	Saved MatchEntity + saved reward (read only)
wallet / shop / achievements / profile / history / settings	Meta screens	-	Ledger, catalog, unlocks, stats, DataStore
leaderboard/{gameId} (new)	Rankings	gameId, scope	Cached top-50 + own rank (cloud)
account (new)	Sign-in, cloud save, delete account	-	Auth state, last sync time

4.2 State pattern used by every screen

```
data class GameplayUiState(
    val phase: MatchPhase = MatchPhase.LOADING,      // LOADING, COUNTDOWN, RUNNING, PAUSED, ENDED
    val board: BoardUi? = null,                        // immutable, @Immutable model
    val scores: List<SeatScoreUi> = emptyList(),
    val activeSeat: Int = 0,
    val inputLocked: Boolean = false,
    val banner: String? = null,
    val connection: ConnectionUi = ConnectionUi.Local
)
sealed interface GameplayEvent { data class CellTapped(val i: Int) : GameplayEvent; data object Pause : GameplayEvent }
sealed interface GameplayEffect { data class GoToResult(val matchId: String) : GameplayEffect; data class Toast(val msg:
    String) : GameplayEffect }

@HiltViewModel
class GameplayViewModel @Inject constructor(private val coordinator: GameCoordinator, saved: SavedStateHandle) : ViewModel() {
    val state: StateFlow<GameplayUiState> = coordinator.observe(saved["matchId"]!!).map(::toUi)
        .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5_000), GameplayUiState())
    private val _effects = Channel<GameplayEffect>(Channel.BUFFERED); val effects = _effects.receiveAsFlow()
    fun onEvent(e: GameplayEvent) = viewModelScope.launch { coordinator.submitLocal(e.toAction()) }
}
```

4.3 Two rendering pipelines

Pipeline	Used for	How it works	Rules
Compose UI	Menus, boards (Tic Tac Toe, Connect Four, Ludo, Chess, Memory, Merge), scoreboards, results	State flow -> recomposition only of changed cells; animate*AsState for disc drops and flips	Keys on lists, @Immutable models, derivedStateOf for computed values
Game canvas loop	Air Hockey, Ping Pong, Pinball, Tank, Sumo, Runner, Sky Jumper, Balloons	One Canvas driven by withFrameNanos; physics in a GameLoop object outside recomposition; draws interpolated state	No allocation per frame, object pools, max 5 physics steps per frame

4.4 Input handling

- **Turn-based:** a tap becomes a GameAction (PlaceMark(4)). The ViewModel sets inputLocked=true until the engine returns the new state or a rejection (150 ms double-tap guard).
- **Split screen:** pointerInput tracks each pointerId; a touch belongs to the zone where it started; its seat id is attached to the action. Up to 4 zones, text rotated 180 degrees (or 90) toward each player.
- **Real-time:** the input sampler stores the latest drag vector per seat; the loop reads it each fixed step. On two phones only the sampled input is sent (30/s), never positions of other objects.

4.5 Design system components (core:designsystem)

Component	Purpose
MvTheme	Light/dark palette from your guide (#7955E8, #FFD84D, #FF6B6B, #4DB9F5, #55D6A5), dynamic colour optional
GameCard, CategoryChip, PlayerBadge	Library grid and player counts; 48 dp touch targets
CoinCounter	Rolls old -> new balance; reads saved value only
SeatZone	Rotated, coloured player area with its own score
CountdownOverlay, PauseSheet, ResultScene	Shared lifecycle UI for all 28 games
ConnectionPill	Local / Wi-Fi / Bluetooth / Online and ping, shown in Lobby and Gameplay
ReducedMotion wrapper	Turns confetti and shakes off when the setting is on

5. Application layer - use cases and coordinators

The application layer is the 'backend inside the phone'. It is where the engine, the database and the network meet. Each use case does one job and is unit-tested with fakes.

Use case	Input	Does	Writes
CreatePlayerUseCase	nickname, avatarId	Creates player + wallet + starter claim	Player, Wallet, Ledger (1 txn)
StartMatchUseCase	gameId, config, seats, mode	Validates player count/mode, creates session, seed, state machine CREATED	Match (CREATED), MatchPlayers
SubmitActionUseCase	matchId, action, playerId	State-machine check, engine.validate, engine.reduce, emit event	MatchEvent (turn games), Snapshot on turn change
PauseResumeUseCase	matchId	RUNNING <-> PAUSED, stops timers	Snapshot
FinishMatchUseCase	matchId	Gets outcome, marks COMPLETED/DRAW, triggers reward	Match, MatchPlayers (1 txn)
AwardRewardsUseCase	matchId	RewardCalculator -> claims -> ledger + wallet	Ledger, Wallet, Outbox (1 txn)
PurchaseItemUseCase	itemId, claimId	Checks balance >= price, owned?, spends	Ledger (negative), Inventory, Wallet, Outbox
RestoreMatchUseCase	-	On launch, finds RUNNING/PAUSED match with valid snapshot	Match -> RECOVERING or ABANDONED
HostRoomUseCase / JoinRoomUseCase	transport, code	Discovery, handshake, config exchange	Session in RAM
SyncUseCase (new)	-	Push outbox, pull remote changes, merge	Outbox status, cursors, merged rows
SubmitScoreUseCase (new)	matchId	Builds signed score claim for leaderboard	Outbox

5.1 GameCoordinator - the conductor

GameCoordinator is a session-scoped object (one per running match). It owns the engine instance, the current state in RAM, the state machine, the timer job and the transport. Every input - from a local finger, a guest phone, or the server - enters through the same door:

```
class GameCoordinator<S : Any, A : Any>(
    private val game: MiniGame<S, A>, private val machine: MatchStateMachine,
    private val transport: MultiplayerTransport, private val role: Role,          // HOST, GUEST, LOCAL
    private val repo: MatchRepository, private val scope: CoroutineScope          // session scope
) {
    private val _state = MutableStateFlow(game.createInitialState(config, players))
    suspend fun submit(action: A, playerId: String) {
        if (machine.current != RUNNING) return reject(NotRunning)
        when (role) {
            Role.GUEST -> transport.send(actionMessage(action, playerId))      // host decides
            else -> {
                val v = game.validateAction(_state.value, action, playerId)
                if (v !is Valid) return reject(v)
                val next = game.reduce(_state.value, action, playerId)
                _state.value = next
                if (role == Role.HOST) transport.send(eventMessage(action, playerId, next))
                repo.checkpointIfNeeded(matchId, next)                          // only on turn change
                if (game.getOutcome(next) !is Incomplete) finish()
            }
        }
    }
}
```

5.2 Threads and dispatchers

Thread / dispatcher	Runs	Never runs
Main	Compose, ViewModel state updates, turn-based reduce (microseconds)	DB, sockets, maze generation, AI
Frame clock (withFrameNanos)	Physics fixed-step loop + drawing for arcade games	Allocation, DB, logging
Dispatchers.Default	Chess AI, maze DFS/BFS, word-list parsing, seed generation	UI
Dispatchers.IO	Room, DataStore, asset reads, socket read/write, HTTP	Game rules
WorkManager	Background cloud sync with network constraint	Gameplay

6. The game engine in depth

6.1 Contract (from your guide, extended)

```
interface MiniGame<S : Any, A : Any> {
    val definition: GameDefinition                    // id, name, category, minPlayers, maxPlayers, modes, family
    val actionSerializer: KSerializer<A>              // NEW: needed for network + replay
    val stateSerializer: KSerializer<S>               // NEW: snapshots, resync, server
    fun createInitialState(config: GameConfig, players: List<GamePlayer>, seed: Long): S   // NEW: seed
    fun validateAction(state: S, action: A, playerId: String): ActionValidation
    fun reduce(state: S, action: A, playerId: String): S                                   // pure, deterministic
    fun systemActions(state: S, nowTick: Long): List<A> = emptyList()   // NEW: timers, spawns, dice (host only)
    fun getOutcome(state: S): GameOutcome
    fun publicView(state: S, viewer: String): S = state   // NEW: hide secrets (words, RPS picks, bomb timer)
}
```

Four additions make the contract work across phones and servers: serializers (so any state/action can travel), an explicit **seed**, **systemActions** for things the rules do on their own (timer expiry, dice roll, mole spawn) and **publicView** so the host only ever sends each player what they are allowed to see.

6.2 Registry and plug-in model

```
@Module @InstallIn(SingletonComponent::class)
object TicTacToeModule { @Provides @IntoSet fun game(): MiniGame<*, *> = TicTacToeGame() }

class GameRegistry @Inject constructor(games: Set<@JvmSuppressWildcards MiniGame<*, *>>) {
    private val byId = games.associateBy { it.definition.id }
    fun find(id: String): Result<MiniGame<*, *>> = byId[id]?.let { Result.success(it) } ?: Result.failure(GameNotFound)
}
```

Adding game 29 = one new package + one @IntoSet provider. Home, Library, Setup, Result and Wallet need no changes.

6.3 Match state machine



Side states: RUNNING -> PAUSED -> RUNNING; RUNNING -> DISCONNECTED -> RECOVERING -> RUNNING; any non-terminal -> ABANDONED. One central `canTransition(from, to)` table, unit-tested for every allowed and forbidden pair. A new 'RESULT_SAVED' terminal marker confirms the reward transaction committed.

6.4 Randomness without breaking determinism

```
// WRONG: reduce() calls Random.nextInt(6)  -> host and guest roll different dice
// RIGHT: host creates the random value and it becomes part of the action/event
val rng = SplittableRandom(session.seed xor turnIndex)            // host only
val roll = DiceRolled(seat = 2, value = 1 + rng.nextInt(6))      // system action
state = game.reduce(state, roll, SYSTEM)                          // every device applies the same value
```

Shuffles (Memory), maze layout, word order and balloon spawns use **the session seed**, which the host sends once in MATCH_CONFIG. A full match can then be replayed from (seed + action log) - this is also how the server verifies leaderboard scores (section 10.4).

6.5 Fixed-step physics loop

```
const val STEP_NS = 16_666_667L            // 1/60 s
var acc = 0L; var last = 0L
LaunchedEffect(matchId) { while (isActive) withFrameNanos { now ->
    if (last != 0L) acc += min(now - last, 250_000_000L)   // clamp after pauses
    last = now
    var steps = 0
    while (acc >= STEP_NS && steps < 5) { world.step(inputs.snapshot()); acc -= STEP_NS; steps++ }
    renderAlpha = acc.toFloat() / STEP_NS                 // canvas draws lerp(prev, curr, alpha)
} }
```

6.6 Engine per game family

Family	State lives	Who advances	Saved to Room	Network
Turn-based (Ludo, S&L, TTT, Connect 4, Dots, Chess, Memory, Word)	RAM	Player actions + host system actions (dice)	Every turn change + result	Action -> host -> event
Real-time physics (Air Hockey, Pong, Football, Sumo, Tank, Fall)	RAM (world)	Fixed 60 Hz loop on host	Result only	Inputs 30/s up, snapshots 20/s down
Reaction/timing (Reaction, Whack, Balloon, Simon, Bomb)	RAM	Host scheduler + taps	Result	Scheduled events with host timestamps
Score challenge (Pinball, Runner, Jumper, Merge, Color Panic)	RAM	Each phone alone	Checkpoint (Merge) + result	Final score + seed only

7. Database and local storage

Storage is split exactly as your guide says - RAM for live play, Room for records, DataStore for settings, a bounded disk cache for assets - with two additions: an **Outbox** table for cloud sync and an **Inventory** table for shop items.

7.1 Entity relationship overview

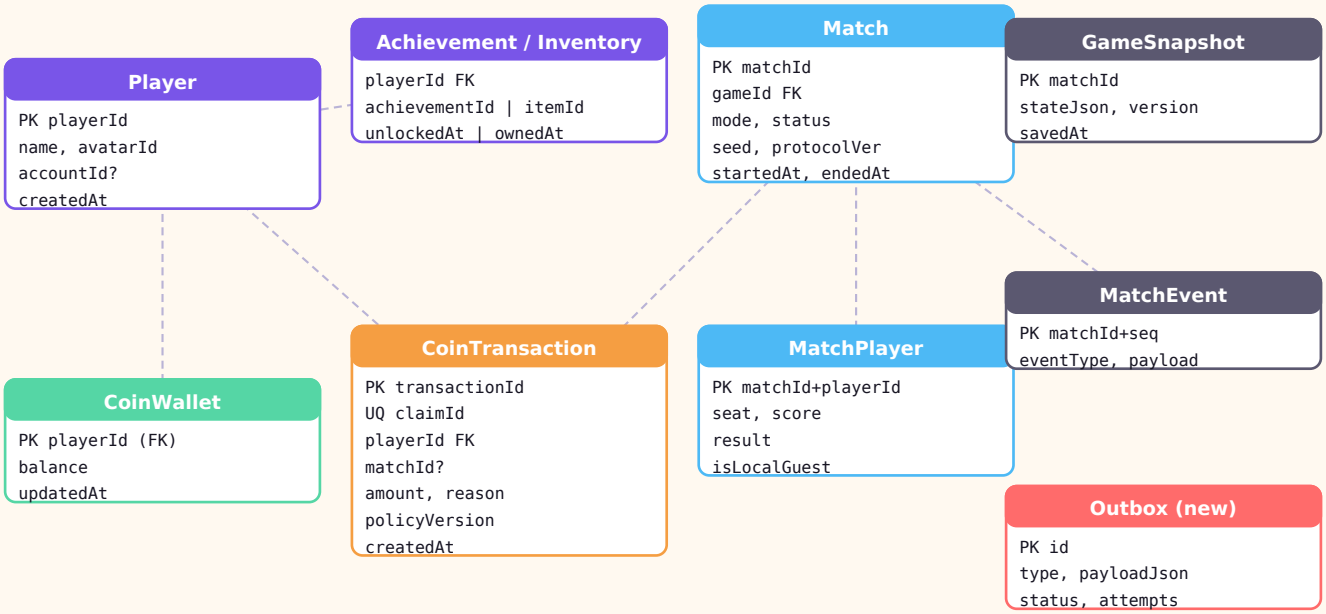


Figure 7.1 - Main Room entities (dashed lines = foreign keys). GameEntity, Favorite, GameProgress and Tournament tables as in your guide.

7.2 Table definitions

Table	Columns (type)	Keys / indices	Written when
player	playerId TEXT, name TEXT, avatarId TEXT, accountId TEXT?, createdAt INT	PK playerId	Onboarding, profile edit, account link
coin_wallet	playerId TEXT, balance INT, updatedAt INT	PK, FK -> player	Inside every reward/purchase txn only
coin_transaction	transactionId TEXT, claimId TEXT, playerId, matchId?, amount INT, reason TEXT, policyVersion INT, createdAt, syncedAt?	PK transactionId, UNIQUE claimId, IDX playerId+createdAt	Reward, purchase, starter grant, cloud merge
match	matchId, gameId, mode, status, seed INT, protocolVersion, startedAt, endedAt?	PK, IDX gameId, IDX status	Create, state changes, finish
match_player	matchId, playerId, seat INT, score INT, result TEXT, isLocalGuest BOOL	PK (matchId, playerId), FK	Create, finish
game_snapshot	matchId, stateJson TEXT, version INT, savedAt	PK matchId (one latest)	Turn change, pause, background
match_event	matchId, sequence INT, eventType, payload	PK (matchId, sequence)	Accepted events (turn games, replays)
achievement	achievementId, playerId, progress INT, unlockedAt?	PK (achievementId, playerId)	After match finish
inventory (new)	itemId, playerId, ownedAt, equipped BOOL	PK (itemId, playerId)	Purchase txn

Table	Columns (type)	Keys / indices	Written when
outbox (new)	id, type, payloadJson, createdAt, attempts INT, status	PK id, IDX status	Any change that must reach the cloud
game_progress	gameId, playerId, bestScore, medals, plays	PK (gameId, playerId)	Finish (score games)

7.3 The corrected reward transaction

```
@Transaction suspend fun AppDb.awardClaims(claims: List<RewardClaim>) = withTransaction {
    for (c in claims) {                                  // c.claimId = "$matchId:$playerId:$reason"
        if (c.amount <= 0) continue
        val inserted = coinDao.insertIgnore(c.toEntity()) // returns -1 if claimId already exists
        if (inserted == -1L) continue                     // already paid -> skip, no error
        walletDao.add(c.playerId, c.amount)
        outboxDao.insert(OutboxEntity.ledger(c))          // cloud will learn about it later
    }
    matchDao.markResultSaved(claims.first().matchId)
}
```

This keeps your rule (one match can never pay twice) while allowing a single run to earn 'level', 'gold' and 'personal_best' as three separate, individually-protected claims.

7.4 DataStore keys and cache

Store	Keys / contents	Limit
settings (Preferences DataStore)	sound, vibration, theme, reducedMotion, language, lastPlayerId, cloudSyncEnabled	Tiny
session (Preferences, encrypted values)	accessToken expiry, deviceId, syncCursor (refresh token in Android Keystore-encrypted file)	Tiny
remote_config (Proto/JSON file)	Last fetched config + ETag, feature flags, min app version	< 50 KB
L2 asset cache	Thumbnails, downloadable themes, word packs	Bounded, e.g. 40 MB LRU
L1 RAM	Live state, decoded bitmaps, SoundPool	Bitmap LRU e.g. 1/8 of app memory class

7.5 Migrations and recovery

- exportSchema = true; every schema version checked into git; MigrationTestHelper test for each version step.
- Never fallbackToDestructiveMigration on player data. Version 2 of this plan adds outbox + inventory + claimId.
- Snapshots carry a version. On restore: unknown or corrupt version -> discard snapshot, match ABANDONED, no crash.

8. Local phone-to-phone connectivity

8.1 Transport layer

All transports implement your MultiplayerTransport interface. A new **TransportSelector** picks the best available one automatically (your recommended 'Adaptive' choice):

Order	Transport	Chosen when	Typical use
1	Wi-Fi Direct (TCP sockets over P2P group)	Both phones support Wi-Fi P2P and permission granted	Real-time physics, 2-4 players
2	Nearby Connections (optional)	Google Play services present on all phones	Easy discovery fallback
3	Bluetooth Classic RFCOMM	Wi-Fi P2P failed or unavailable	Turn-based games only
4	Online WebSocket (new)	Internet available and player chooses 'Play online'	Anywhere in the world
-	LocalPassPlayTransport	Same device	Loops messages back in memory

Real-time physics games require Wi-Fi Direct, Nearby or Online; the lobby greys them out on Bluetooth.

8.2 Room handshake protocol

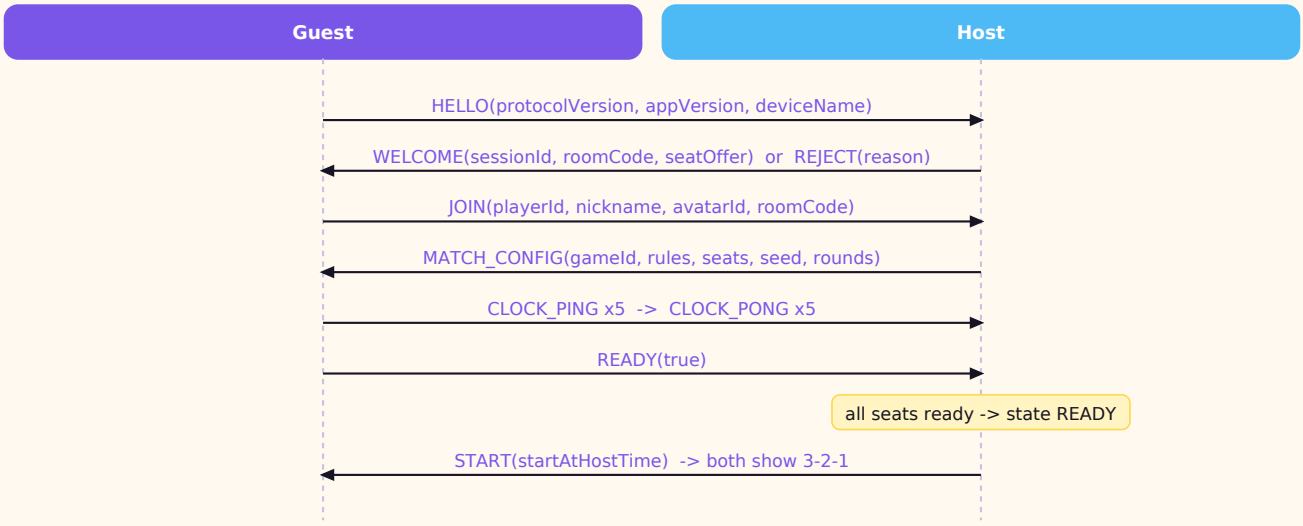


Figure 8.1 - Every two-phone match starts with this exchange, on any transport.

8.3 Message envelope and catalogue

```
@Serializable data class GameMessage(
    val protocolVersion: Int, val sessionId: String, val matchId: String?,
    val senderId: String, val messageId: String, val sequence: Long, val ackUpTo: Long,  // ackUpTo NEW
    val type: MsgType, val gameId: String?, val tick: Long? = null, val payload: JsonElement
)   // framing on sockets: [4-byte length][JSON or CBOR bytes], max 16 KB per frame
```

Type	Direction	Purpose
HELLO / WELCOME / REJECT / JOIN	both	Version check, admission
MATCH_CONFIG / READY / START	host -> guest, guest -> host	Setup and synchronized start
PLAYER_ACTION	guest -> host	Turn move or real-time input sample

Type	Direction	Purpose
EVENT_ACCEPTED / ACTION_REJECTED	host -> all / host -> sender	Authoritative result of an action
SNAPSHOT	host -> guests	Full public state (resync, late join, 20 Hz for physics)
CLOCK_PING / CLOCK_PONG	both	Clock offset and RTT
HEARTBEAT	both, every 1 s	Detect disconnect after 3 s silence
PAUSE / RESUME / FORFEIT	both	Lifecycle
RESYNC_REQUEST	guest -> host	Ask for snapshot + missed events after a gap in sequence
MATCH_RESULT / BYE	host -> all	Final outcome (each device pays its own local players)

8.4 Turn-based action flow on two phones

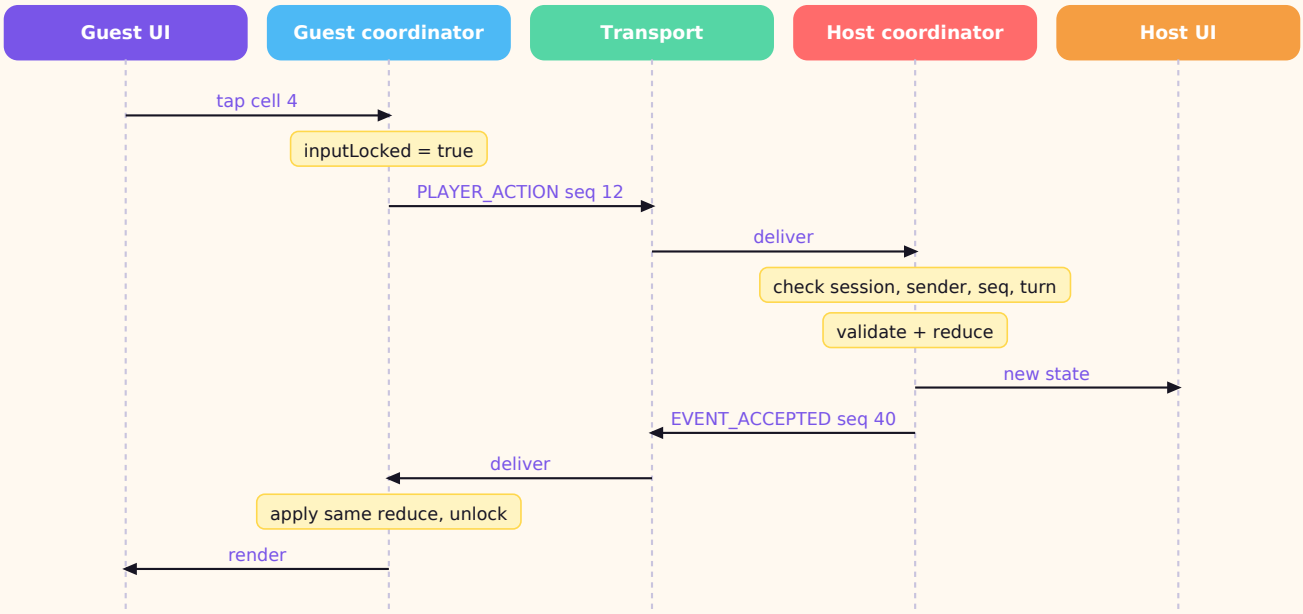


Figure 8.2 - The guest never changes its own state until the host accepts the action.

8.5 Clock sync for reaction games

```
// guest sends t0 (own clock); host replies with t1 (host clock); guest receives at t2
rtt    = t2 - t0
offset = t1 - (t0 + rtt / 2)                 // hostTime = guestTime + offset
// do 5 pings, keep the sample with the smallest rtt; repeat every 10 s during the match
// guest tap report: tapHostTime = tapGuestTime + offset  -> host compares all taps on ONE clock
```

8.6 Hidden information (Rock Paper Scissors, Guess the Word, Bomb Squad)

- **Commit-reveal** for RPS: each device sends COMMIT(sha256(choice + 16-byte salt)); after both commits arrive, both send REVEAL(choice, salt); everyone verifies the hash. Neither side can change or peek.
- **publicView()** strips secrets before any SNAPSHOT is sent (word replaced by mask, bomb timer by 'tick' level).
- Offline we accept that a modified host app could cheat - coins are cosmetic. Online, the **server** is the host, so secrets never reach any player's phone.

8.7 Failure handling

Situation	Detection	Behaviour
Guest silent 3 s	Missing heartbeats	Match -> DISCONNECTED, game paused for all, 60 s reconnect timer
Guest returns	HELLO with same sessionId + playerId	Host sends SNAPSHOT + events after ackUpTo -> RECOVERING -> RUNNING
Guest timer expires	60 s	Turn games: guest forfeits if match over 30% done, else ABANDONED
Host disappears	Heartbeat loss on guest	Interrupted, no coins (host migration = later feature)
Out-of-order / duplicate	sequence <= last seen	Dropped silently; gap -> RESYNC_REQUEST
Version mismatch	HELLO protocolVersion	REJECT('update app') with friendly screen
Flooding / huge packet	> 60 msgs/s or > 16 KB	Drop and, if repeated, disconnect peer

8.8 Permissions by Android version

Android	Bluetooth	Wi-Fi Direct / nearby	When to ask
7 - 11 (API 24-30)	BLUETOOTH, BLUETOOTH_ADMIN	ACCESS_FINE_LOCATION (+ location services on)	Only when the user taps Create/Join Room
12 (API 31-32)	BLUETOOTH_SCAN (neverForLocation), BLUETOOTH_CONNECT, BLUETOOTH_ADVERTISE	ACCESS_FINE_LOCATION for Wi-Fi P2P	Same, with rationale sheet
13+ (API 33+)	Same as 12	NEARBY_WIFI_DEVICES (neverForLocation)	Same; no location needed

Always check the current Play policy and Android docs before release - permission rules change between versions.

9. Server backend - connectivity to the cloud

Principle: the cloud is an add-on, never a requirement

Every feature in your guide keeps working with airplane mode on. The server adds: backup across devices, global leaderboards, online matches with friends far away, and the ability to update content and settings without a new APK.

9.1 What the server does (and does not do)

Service	Responsibility	Phase
Auth	Anonymous device account, optional Google sign-in link, JWT access + rotating refresh tokens, account deletion	Cloud 1
Profile + Cloud save	Nickname, avatar, owned items, achievements, wallet ledger, match history summaries	Cloud 1
Sync	Accepts outbox batches, returns changes since cursor, idempotent by claimId / eventId	Cloud 1
Remote config + content	Feature flags, reward policy version, min app version, word packs, themes manifest	Cloud 1
Leaderboards	Per-game, weekly + all-time, verified scores only	Cloud 2
Matchmaking + rooms	Private room codes over internet, quick-match queue by game and player count	Cloud 2
Game server	Authoritative online matches running core:gameengine, WebSocket sessions	Cloud 2
Moderation + reports	Nickname filter, report player, ban list	Cloud 2
Not done by server	Offline matches, local multiplayer, rendering, pass-and-play	-

9.2 Technology options

Option	Pros	Cons	Verdict
A. Kotlin Ktor + PostgreSQL + Redis (containers)	Same language as app; run the same engine module on server; full control; portable	You run and secure it	Recommended
B. Firebase (Auth, Firestore, Functions)	Very fast to start, managed scaling	Cannot easily host your Kotlin engine for real-time play; adds Play-services size; vendor lock-in	Good for prototypes only
C. Hybrid: Firebase Auth + Ktor services	Managed sign-in, own game server	Two systems to learn	Valid alternative

9.3 Server architecture

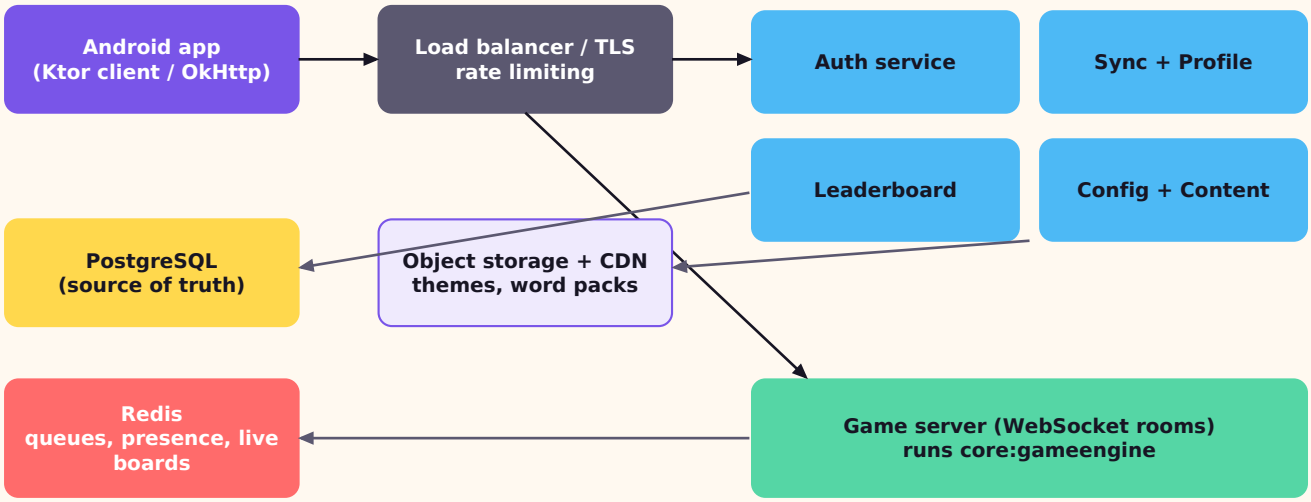


Figure 9.1 - Start as a single Ktor service with modules (a 'modular monolith'); split only when load demands it.

9.4 REST and WebSocket API (v1)

Method + path	Body / params	Returns	Notes
POST /v1/auth/device	deviceId, appVersion	accessToken (15 min), refreshToken, userId	Anonymous account; no personal data
POST /v1/auth/google	Google ID token	tokens, linked userId	Server verifies token; merges anonymous account
POST /v1/auth/refresh	refreshToken	new pair	Rotation; reuse detection revokes family
DELETE /v1/account	-	202	Required in-app account deletion; also via web link
GET / PUT /v1/profile	nickname, avatarId, equipped	profile	Nickname filtered server-side
POST /v1/sync/push	batch of outbox items (max 200)	per-item status: accepted | duplicate | rejected	Idempotent by claimId / eventId
GET /v1/sync/pull?cursor=	cursor	changes, nextCursor	Ledger, inventory, achievements from other devices
GET /v1/config	If-None-Match	flags, rewardPolicy, minVersion	Cached on phone; offline uses last copy
GET /v1/content/manifest	-	list of packs + hashes + URLs	Downloaded lazily, hash-verified
GET /v1/leaderboards/{gameId}	scope=weekly|all, around=me	top N + own rank	Served from Redis sorted set
POST /v1/scores	gameId, score, seed, inputLogHash, integrityToken	accepted rank or review	See 10.4
POST /v1/rooms	gameId, maxPlayers	roomCode, wsTicket	Private online room
POST /v1/rooms/{code}/join	-	wsTicket	Room full / expired errors
POST /v1/matchmaking/queue	gameId, players	ticketId	Paired via Redis; result pushed over WS

Method + path	Body / params	Returns	Notes
WS /v1/play?ticket=	GameMessage frames	GameMessage frames	Same envelope as local play
POST /v1/reports	targetUserId, reason, matchId	202	Moderation queue
GET /health, /ready	-	200	For the load balancer

10. Server data, sync and security

10.1 PostgreSQL schema

Table	Key columns	Constraints / indices
users	id UUID, created_at, status (active/banned/deleted), google_sub?	UNIQUE google_sub
devices	id, user_id, device_id_hash, app_version, last_seen	IDX user_id
refresh_tokens	id, user_id, family_id, token_hash, expires_at, revoked_at	IDX family_id
profiles	user_id, nickname, avatar_id, frame_id, updated_at	PK user_id
ledger_entries	id, user_id, claim_id, amount, reason, match_id?, policy_version, client_created_at, server_received_at	UNIQUE (user_id, claim_id)
wallets	user_id, balance (derived = SUM ledger), version	Updated in same txn as ledger insert
inventory	user_id, item_id, acquired_at	PK (user_id, item_id)
achievements	user_id, achievement_id, progress, unlocked_at	PK (user_id, achievement_id)
matches	id, game_id, mode (local/online), seed, status, started_at, ended_at	IDX game_id, ended_at
match_players	match_id, user_id, seat, score, result	PK (match_id, user_id)
scores	id, user_id, game_id, score, seed, input_log_hash, verified (bool), created_at	IDX (game_id, score DESC)
rooms	code, game_id, host_user_id, status, expires_at	PK code, TTL cleanup job
reports	id, reporter_id, target_id, reason, match_id, status	IDX status
app_config	key, json_value, version	PK key

10.2 Offline-first sync (outbox pattern)

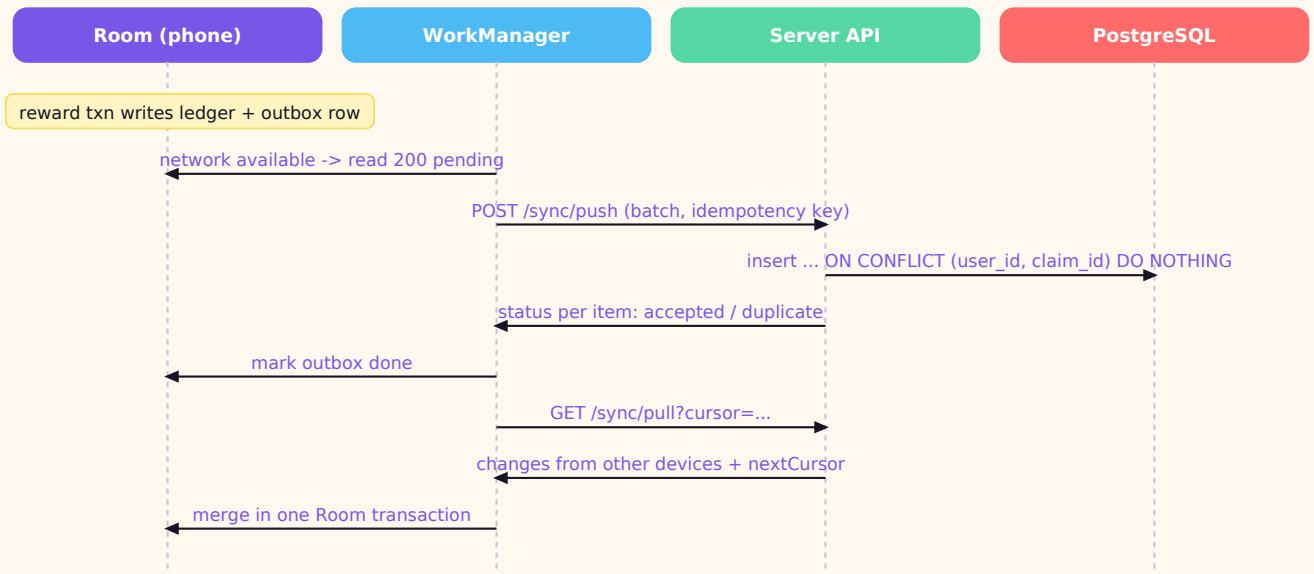


Figure 10.1 - Sync is safe to repeat: every item has an id, so retries never double-pay.

Conflict rules

Data	Rule
Ledger (coins)	Append-only union by claimId. Balance is always recomputed from the ledger, never copied.
Spending on two offline devices	Server applies spends in received order; if a spend would make balance negative it still records the item as owned and adds an 'adjustment' entry flooring balance at 0 (items are cosmetic - no player is punished).
Inventory / achievements	Union (once owned or unlocked, always owned).
Profile fields	Last-writer-wins by updated_at.
Match history	Union by matchId; local-only for guest seats.
Settings	Never synced (device-specific).

10.3 Online match flow (server is the host)

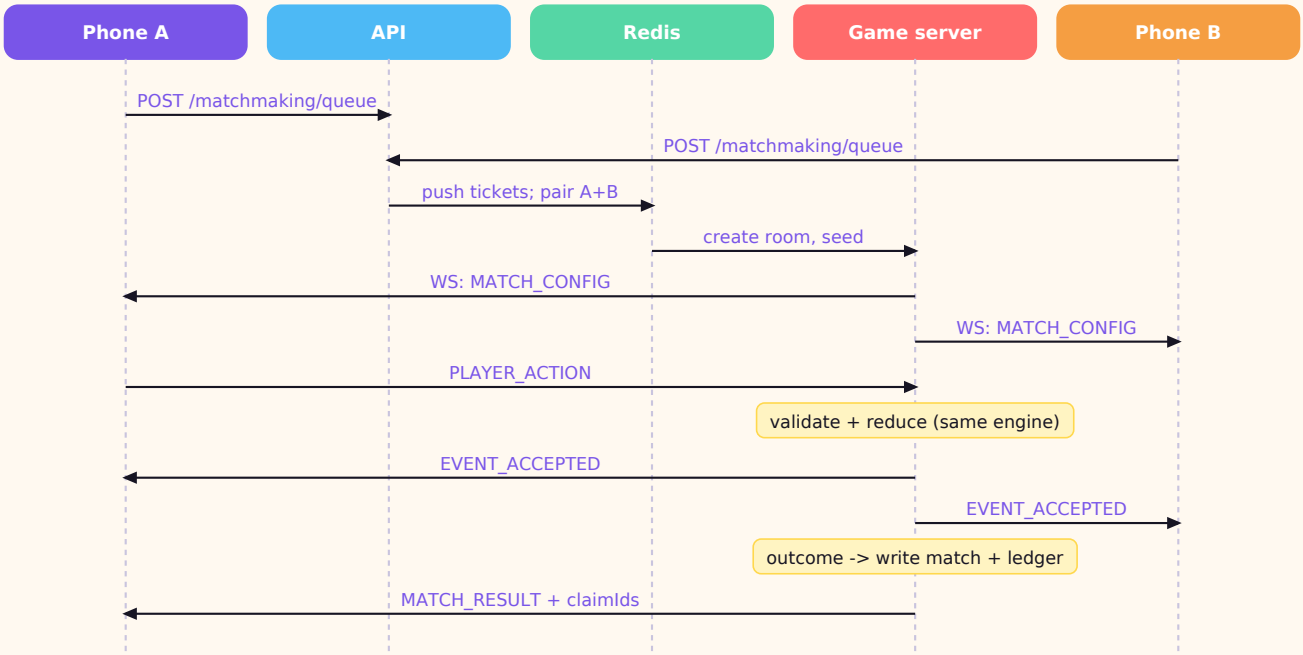


Figure 10.2 - Online, coins are written by the server and pulled down by sync, not by the phone.

10.4 Score verification for leaderboards

- Score games are deterministic from (seed + input log). The phone uploads score, seed, a compact input log (or its hash plus the log on request) and a **Play Integrity** token.
- Server checks: plausible score for play time, integrity verdict, rate limits. Top-100 scores are **re-simulated** with the shared engine before being shown publicly. Unverified scores are visible only to their owner.

10.5 Security checklist

Area	Measure
Transport	HTTPS/WSS only (TLS 1.2+), no cleartext traffic in network security config
Tokens	Short-lived JWT access tokens; refresh token hashed on server, encrypted with Android Keystore on phone; rotation + reuse detection
Input	Every request validated with schema; max body size; server never trusts client balance, score or result

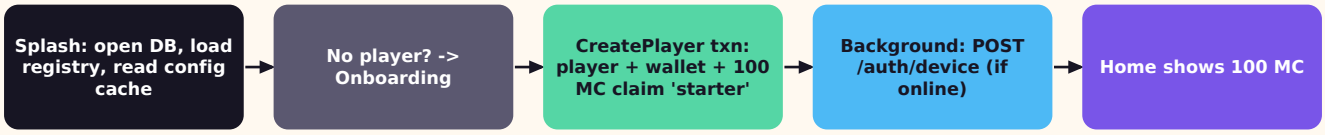
Area	Measure
Abuse	Per-user and per-IP rate limits in Redis; nickname filter; report and ban
Data	Minimal personal data (no email unless Google link); encrypted backups; deletion within a defined window
Secrets	No API secrets inside the APK; environment-based config on server
Play policy	Data safety form, privacy policy URL, in-app and web account deletion; if children are a target audience, follow the Families policy

10.6 Deployment and operations

Item	Plan
Packaging	Ktor fat-jar in a Docker image; one image, env vars per environment
Environments	dev (local Docker Compose: app + Postgres + Redis), staging, production
Hosting	Any container platform or a small VPS to start; managed Postgres with daily backups + point-in-time recovery
Migrations	Flyway SQL migrations run on deploy
CI/CD	GitHub Actions: tests -> build image -> deploy to staging -> manual approve -> production
Observability	Structured logs, metrics (request rate, errors, p95 latency, WS rooms), alerts on error spikes
Kill switch	Remote config flags can disable online play or a single game without a new APK

11. End-to-end flows - the whole app working

11.1 First launch



11.2 A same-device Tic Tac Toe match, tap to coins

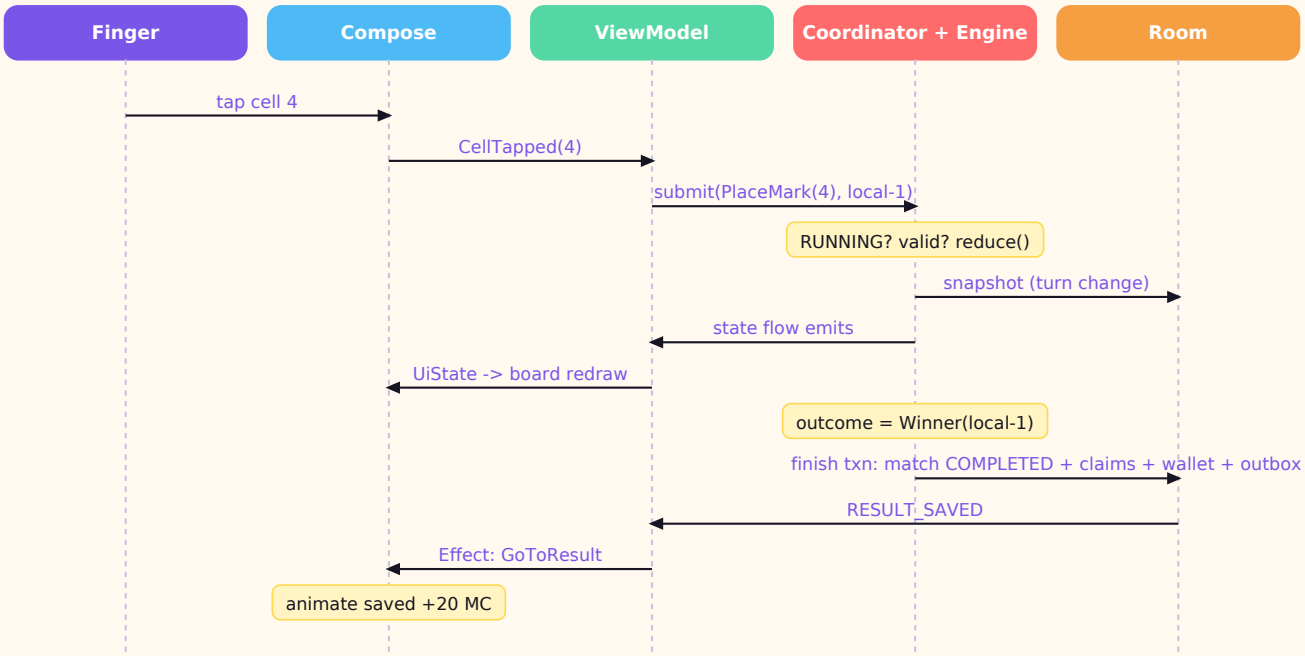
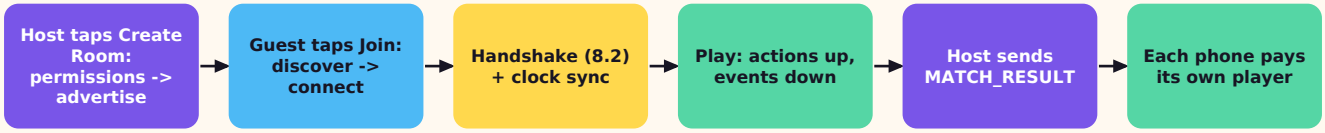
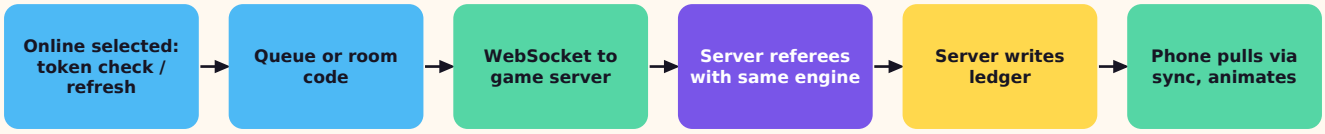


Figure 11.1 - The animation only plays after the database confirms the reward.

11.3 Two phones over Wi-Fi Direct



11.4 Online match



11.5 Buying an item in the Shop



11.6 App killed in the middle

Moment of kill	On next launch
During a turn-based match	RestoreMatchUseCase finds snapshot -> 'Resume match?' -> RECOVERING -> RUNNING
During a physics match	Match marked ABANDONED, 0 coins (no snapshot by design)
After reward txn, during animation	Ledger already has claims -> result screen shows saved values, no new payout
Before reward txn committed	Match is still RUNNING/finished-unsaved -> FinishMatchUseCase re-runs; claimIds make it safe
During cloud sync	Outbox items still pending -> sent again; server dedupes

12. Project structure (client + server)

```
MiniVerse/                                (one Gradle build, Kotlin everywhere)
  app/                    MainActivity, MiniVerseApplication, NavHost, Hilt setup
  build-logic/            convention plugins (android-library, compose, hilt, kotlin-jvm)
  core/
    model/                ids, GamePlayer, GameDefinition, GameMessage     <- pure Kotlin (shared with server)
    gameengine/           MiniGame, Registry, StateMachine, Rng, Outcome   <- pure Kotlin (shared with server)
    designsystem/ ui/     theme, components
    database/             Room entities, DAOs, migrations
    datastore/ cache/     settings, session, asset cache
    economy/              RewardCalculator, claims, wallet repository
    multiplayer/          transport interface, protocol, clock sync, selector
    connectivity/         WifiDirect, Bluetooth, Nearby, PassPlay transports
    network/   (new)      HTTP client, auth interceptor, API DTOs, WebSocket transport
    sync/      (new)      outbox, SyncWorker, merge logic
    audio/ animation/ testing/
  feature/                splash onboarding home library details setup lobby gameplay result
                          wallet shop achievements profile history settings leaderboard account
  games/                  tictactoe connectfour snakes memory reaction rps airhockey pinball ...
  server/      (new)
    app/                  Ktor main, routing, plugins (auth, rate limit, serialization)
    auth/ profile/ sync/ leaderboard/ config/ matchmaking/ gameserver/
    db/                   Exposed or jOOQ repositories, Flyway migrations
  games-rules/ (new)      pure-Kotlin rule modules extracted from games/ so the server can load them
```

Area	Client	Server
Language	Kotlin	Kotlin (JVM)
UI / framework	Jetpack Compose + Material 3	Ktor
DI	Hilt	Koin or manual constructor wiring
Data	Room, DataStore	PostgreSQL (Exposed/jOOQ), Redis, Flyway
Serialization	Kotlinx Serialization	Kotlinx Serialization (same DTOs)
Networking	Ktor client or Retrofit + OkHttp, WebSocket	Ktor server + WebSockets
Background	WorkManager	Coroutines, scheduled jobs
Testing	JUnit, Turbine, Compose UI tests, Robolectric	JUnit, Testcontainers (real Postgres/Redis)

Use the latest stable, mutually compatible versions through the version catalog (libs.versions.toml); check each library's minimum Android API against your API 24 baseline before adding it.

> **Split games into 'rules' and 'renderer'**
>
> Each game becomes two parts: **rules** (state, actions, reduce - pure Kotlin) and **renderer** (Compose/Canvas). The server only loads the rules. This is a small change now and saves a rewrite later.

13. Performance, testing and release quality

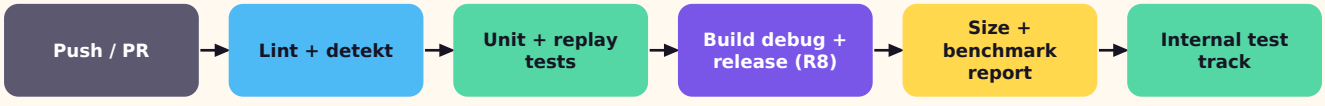
13.1 Budgets

Metric	Target	How to measure
Cold start	< 2 s on a 2 GB phone	Macrobenchmark + Baseline Profile
Screen transition	< 300 ms	Macrobenchmark frame timing
Turn input to redraw	< 100 ms	Trace markers around submit()
Arcade frame rate	60 FPS, no frame > 32 ms	JankStats in debug builds
Local action round trip	< 50 ms Wi-Fi Direct, < 150 ms Bluetooth	PING/PONG logs
Online round trip	< 150 ms same region	Server metrics + client ping
APK / download size	< 25 MB	Build size report in CI
Crash-free sessions	> 99.5%	Crash reporting (if adopted and disclosed)
Server API p95	< 200 ms	Server metrics

13.2 Test pyramid

Level	What	Tooling
Unit (most)	Every game's validate/reduce/outcome, state machine table, reward calculator, claimIds, clock-offset maths, conflict merge	JUnit, property-based tests for reduce determinism
Replay tests	Run 1,000 random seeded matches per game; no exception, always terminates, same result twice	JUnit
Integration	Room DAOs + migrations, reward txn under concurrency, two in-memory transports talking, outbox sync with fake server	Robolectric, MigrationTestHelper, Turbine
Server	API routes, idempotent sync, auth rotation, matchmaking pairing, online room with two fake clients	Ktor testApplication, Testcontainers
UI	Navigation, setup, result, small screen + large font + landscape	Compose UI tests
Manual device lab	Real Wi-Fi Direct and Bluetooth pairs, low-RAM phone, 120 Hz phone	Section 13 device matrix from your guide

13.3 CI pipeline



14. Step-by-step roadmap

Estimate for one developer working steadily. Offline v1.0 is reachable in about 20 weeks; the cloud adds about 10-12 weeks. Every phase ends with a milestone you can demo - do not start the next phase until it passes.

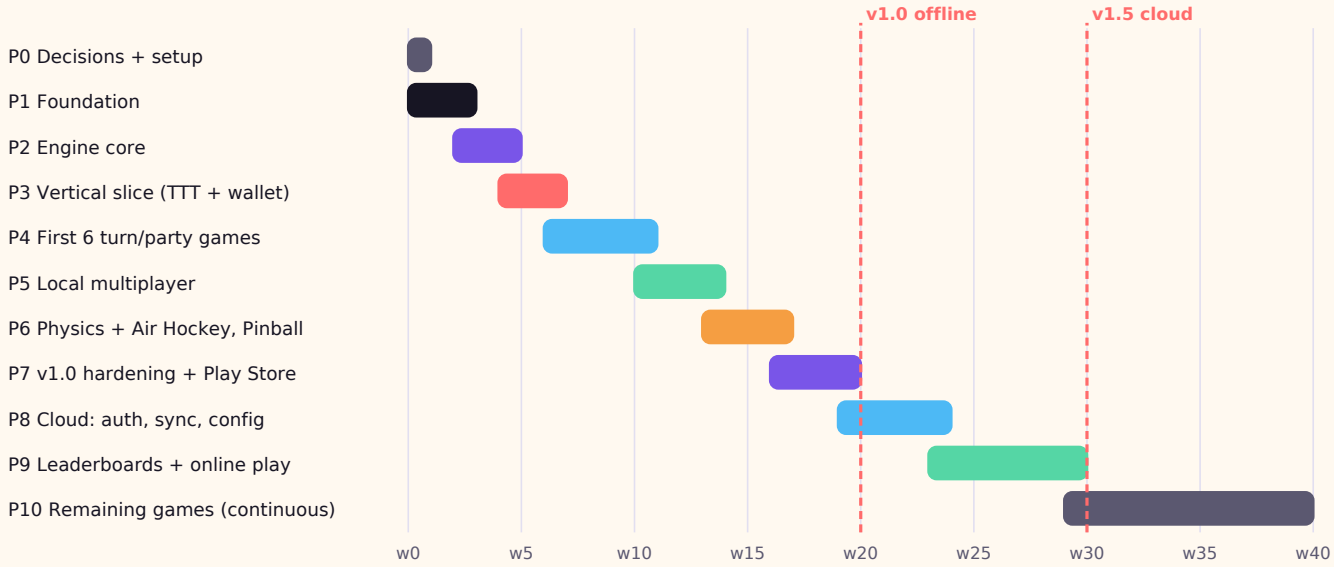


Figure 14.1 - Roadmap timeline (weeks). Phases overlap slightly where polish continues.

Phase	Weeks	Build	Exit test (milestone)
P0 Decisions	1	Lock section 15 decisions, package name, repo, CI skeleton	Decisions written down
P1 Foundation	1-3	Gradle + version catalog, Hilt, theme, nav shell, Room v1, DataStore, Splash, Onboarding, Home, Library	App launches offline, no crash, Library shows registry
P2 Engine	3-5	Contracts, registry, seeded RNG, state machine, coordinator, snapshot serializer, PassPlay transport, dummy game	Dummy game + 100% state-machine tests
P3 Vertical slice	5-7	Tic Tac Toe complete + Result + claim-based reward txn + Wallet + History	Kill app anywhere -> no lost or double coins
P4 First games	7-11	Connect Four, Memory Match, Snakes and Ladders, Reaction Battle, Rock Paper Scissors (+ Number Merge, Whack-a-Mole if time)	Replay tests pass for each; split-screen works
P5 Local multiplayer	11-14	Handshake, Wi-Fi Direct, Bluetooth, selector, clock sync, commit-reveal, reconnect, permissions	Two real phones finish 50 matches without desync
P6 Physics	14-17	Fixed-step loop, collision kit, pools, Air Hockey, Pinball, snapshot + interpolation netcode	60 FPS on low-end phone; two-phone Air Hockey smooth
P7 v1.0	17-20	Shop + inventory, achievements, audio, haptics, accessibility, R8, Baseline Profile, Play listing	Release candidate: 8 games, offline
P8 Cloud 1	20-24	Server skeleton, Postgres, auth, outbox sync, remote config, account deletion, Data safety form	Reinstall app -> sign in -> coins restored
P9 Cloud 2	24-30	Leaderboards + verification, online rooms, matchmaking, WebSocket transport, moderation	Two phones in different cities finish a match
P10 Expansion	30+	Ludo, Dots and Boxes, Mini Chess, Ping Pong, Tank, Sumo, Finger Football, rest of 28, tournaments	Each game passes pre-release tracker

14.1 Unified game phase table (fixes issues 7, 8, 10)

Release	Games
v1.0 (8)	Tic Tac Toe, Connect Four, Snakes and Ladders, Memory Match, Reaction Battle, Rock Paper Scissors, Air Hockey, Pinball Master
v1.1	Mini Ludo, Dots and Boxes, Number Merge, Whack-a-Mole, Ping Pong Battle
v1.2	Mini Chess, Quick Math, Word Scramble, Guess The Word, Balloon Pop
v1.3	Finger Football, Mini Sumo, Tank Battle, Crazy Runner, Sky Jumper
v1.4	Simon Says, Color Panic, Fall Challenge, Bomb Squad, Maze Escape

15. Decisions to confirm before coding

#	Decision	Options	Recommended
D1	Starting MiniCoins	0 / 50 / 100 / 250	100 (from your guide)
D2	Match rewards	20/10/5, 30/15/5, 50/20/10	20 / 10 / 5
D3	Local connectivity	Adaptive, Wi-Fi first, Bluetooth first, Nearby first	Adaptive (selector in 8.1)
D4	Package name	com.yourname.miniverse	Decide now - permanent after publishing
D5 (new)	Coins for extra players on the same phone	Guest seats (no wallet) / quick local profiles	Guest seats in v1.0, local profiles in v1.1
D6 (new)	Explicit quit in two-phone match	Always abandoned / forfeit after X%	Forfeit after 30% of the match
D7 (new)	Backend stack	Ktor + Postgres + Redis / Firebase / hybrid	Ktor + Postgres + Redis
D8 (new)	Account model	Anonymous only / anonymous + Google / Google required	Anonymous + optional Google link
D9 (new)	Target audience age	13+ / includes children	Decide early - changes Play policy duties, ads and data rules
D10 (new)	Crash reporting	None / self-hosted / third-party SDK	Decide and disclose in privacy policy

15.1 Your first two weeks

- Write down D1-D10 and the package name.
- Create the Gradle project with build-logic conventions, core:model and core:gameengine as **plain Kotlin JVM** modules (server-ready from day one).
- Implement MvTheme with your palette, Splash, Onboarding, Home and the Library grid from a registry.
- Room v1 with player, coin_wallet, coin_transaction (UNIQUE claimId), match, match_player, outbox.
- State machine + canTransition table with full unit tests.
- Start Tic Tac Toe rules with replay tests before writing any of its UI.

Final word

Your guide already has the right heart: one deterministic, host-authoritative engine. This roadmap keeps that heart and lets it beat in three places - one phone, a group of nearby phones, and the cloud - without rewriting a single game rule. Build it layer by layer, test each milestone on a cheap real phone, and MiniVerse will be both a fun app and a portfolio-grade system.

Appendix - Error codes and glossary

A. Error model (client and server share codes)

Code	Where	User sees
GAME_NOT_FOUND	Registry	This game is not available - back to Library
INVALID_ACTION / NOT_YOUR_TURN	Engine	Silent shake on the cell
SESSION_EXPIRED	Lobby / WS	Room has closed
PROTOCOL_MISMATCH	Handshake	Ask your friend to update MiniVerse
PLAYER_DISCONNECTED	Transport	Waiting for Riya to reconnect... (60 s)
PERMISSION_DENIED	Room setup	Explain why, button to settings
STORAGE_ERROR	Room	Something went wrong saving - your coins are safe, try again
AUTH_REQUIRED / TOKEN_EXPIRED	API	Silent refresh; if it fails, sign-in prompt for cloud features only
RATE_LIMITED	API	Too many requests - try in a minute
UPDATE_REQUIRED	Config	New version needed for online play (offline still works)

B. Glossary

Term	Meaning
Authoritative host	The one device (or server) whose state is the truth; others only send requests
Deterministic reduce	Same state + same action always gives the same next state
Claim id	Unique name for one reward (match:player:reason); stops double payouts
Outbox	Local table of changes waiting to be sent to the cloud
Snapshot	Full serialized game state used for resume or resync
Interpolation	Drawing between two received snapshots so motion looks smooth
Commit-reveal	Send a hash of your secret first, the secret later, so nobody can change or peek
Idempotent	Doing the same request twice has the same effect as doing it once
Modular monolith	One deployable server with clean internal modules - easy to split later